

Q1: What is done to reduce runtime overhead (JITting)?

- **Ahead-Of-Time (AOT) Compilation:** Compiles code to native machine code before execution to bypass runtime JIT completely.
- **ReadyToRun (R2R):** Pre-compiles native images ahead of time to accelerate application startup.
- **Tiered Compilation:** Uses quick, unoptimized compilation at startup, then re-compiles frequently called ("hot") methods with heavy optimizations later.
- **Profile-Guided Optimization (PGO):** Uses application profiling data to optimize compiled assembly for real-world execution paths.
- **Code Caching:** Stores compiled native code in memory to prevent re-jitting previously executed code.

Q2: How do you run "Hello World" using VS Code?

1. Create a new project in the terminal:

```
dotnet new console -o HelloWorld
```

2. Move into the project directory:

```
cd HelloWorld
```

3. Run the program:

```
dotnet run
```

Q3: Write a brief report about .NET Versions, Namespace, .NET Core, and Solution.

1. .NET Versions

- **.NET Framework (2002–2019):** Windows-only framework used for legacy web and desktop applications (versions 1.0 to 4.8).
- **.NET Core (2016–2019):** Cross-platform, open-source rewrite designed for high-performance cloud applications (versions 1.0 to 3.1).
- **Modern .NET (.NET 5+):** The unified platform combining Framework and Core into a single ecosystem (current version is .NET 8 / .NET 9).

2. Namespace

- A logical container used to organize classes, interfaces, and types to avoid naming conflicts in large applications.
- Example syntax: `namespace MyApp.Services { public class UserService { } }`

3. .NET Core

- A lightweight, modular, and cross-platform runtime that allows .NET applications to execute on Windows, Linux, and macOS.
- Features built-in dependency injection, asynchronous processing, and side-by-side version execution.

4. Solution

- A container file (.sln) used by Visual Studio / .NET CLI to group and manage one or more related projects (e.g., Web API, Class Library, Unit Tests).
- Tracks project dependencies, build configurations, and source control settings across the entire codebase.